SPRINT 2 - PROJETO 3D OPENGL E PYTHON

Iluminacao e Sombra em Computacao Grafica

Dicionario Didatico de Variaveis, Matematica Intuitiva e Roteiro de Explicacao

1. Visao Geral: O Desafio da Luz e da Sombra em 3D

Em Computacao Grafica, uma tela de computador e estritamente bidimensional (2D). Quando renderizamos modelos 3D apenas com cores ou texturas puras, os objetos parecem figuras de papel ou desenhos planos. Nao da para distinguir onde comeca ou termina uma aresta ou se uma face esta inclinada.

A Iluminacao e a Sombra sao as duas principais pistas visuais que o cerebro humano usa para deduzir profundidade, volume e contato fisico com o chao. Neste projeto, implementamos uma solucao elegante e compacta ideal para o nivel junior: um modelo de Iluminacao Difusa Direcional (Lambert) somado a uma Luz Ambiente, com funcoes proprias de algebra linear e Sombras Planar Projetadas baseadas na propria malha 3D.

Regra de Ouro da Computacao Grafica Junior

O CPU (Python) cuida da logica, das posicoes e envia ordens. A GPU (Placa de Video e Shaders GLSL) executa os calculos de iluminacao em paralelo para cada pixel desenhado na tela!

2. Dicionario de Variaveis e Funcoes: O Que Cada Uma Faz?

Abaixo estao detalhadas todas as variaveis e funcoes manuais criadas no projeto, seus valores e como explica-las com naturalidade:

usarLuz (Uniform booleano)

Local: mode3D.py / fragmentShad.glsl **Valor: 1 (3D) ou 0 (2D)**

Significado: Interruptor que avisa a GPU se o que esta sendo desenhado agora deve receber calculo de luz ou ser desenhado com cor pura.

Como Explicar: Pense no interruptor de um quarto. No mundo 3D queremos luz ligada (1). Ja para desenhar o menu 2D e icones da barra lateral, desligamos a luz (0) para que os botoes fiquem sempre nitidos e legiveis.

lightDir (vec3)

Local: mode3D.py (0.5, 1.0, 0.4) **Valor: X=0.5, Y=1.0, Z=0.4**

Significado: Vetor tridimensional que define a direcao de onde vem a luz direta (como os raios do Sol).

Como Explicar: Imagine que colocamos o Sol no ceu: Y=1.0 significa bem do alto, X=0.5 um pouco a direita, e Z=0.4 um pouco a frente. Isso cria um contraste bonito em 3 lados de qualquer caixa.

lightColor (vec3)

Local: mode3D.py (0.85, 0.85, 0.80) **Valor: R=0.85, G=0.85, B=0.80**

Significado: Cor e intensidade da luz direta do Sol que atinge os objetos.

Como Explicar: Valores vao de 0.0 (preto) a 1.0 (branco maximo). Escolhemos 0.85 com azul um pouco menor (0.80), gerando uma luz levemente amarelada e quente como a luz solar suave do meio da manha.

ambientColor (vec3)

Local: mode3D.py (0.45, 0.45, 0.50) **Valor: R=0.45, G=0.45, B=0.50**

Significado: Luz ambiente indireta, que simula a luz que rebate nas paredes, no chao e no ceu da sala.

Como Explicar: Sem luz ambiente, as partes dos objetos viradas de costas para o Sol ficariam pretas como breu! A luz ambiente garante que o lado de tras continue visivel com 45% de claridade suave.

Dicionario de Variaveis e Funcoes (Continuacao)

calcularNormal(vec3 A, vec3 B)

Local: fragmentShad.glsl **Valor: A x B (Produto Vetorial)**

Significado: Funcao manual que substitui o cross do GLSL. Multiplica componentes cruzados e retorna um vetor perpendicular a A e B.

Como Explicar: Calcula a perpendicular exata entre duas tangentes da face: vec3(A.y*B.z - A.z*B.y, A.z*B.x - A.x*B.z, A.x*B.y - A.y*B.x). Mostra dominio real da algebra linear sem depender de funcoes prontas!

calcularIluminacao(vec3 A, vec3 B)

Local: fragmentShad.glsl **Valor: A . B (Produto Escalar)**

Significado: Funcao manual que substitui o dot do GLSL. Multiplica cada coordenada correspondente e soma tudo: A.x*B.x + A.y*B.y + A.z*B.z.

Como Explicar: Retorna um unico numero que mede o quanto dois vetores apontam na mesma direcao. Usado tanto para testar visibilidade com a camera quanto para calcular o brilho da luz!

normalizarVetor(vec3 A)

Local: fragmentShad.glsl **Valor: A / comprimento**

Significado: Funcao manual que substitui o normalize do GLSL. Calcula o tamanho por Pitagoras 3D sqrt(x^2 + y^2 + z^2) e divide o vetor por ele.

Como Explicar: Garante que o vetor tenha tamanho exatamente 1.0 (vetor unitario). Conta com protecao contra divisao por zero (comprimento < 0.000001) para nao travar a GPU!

N (Normal da Superficie)

Local: fragmentShad.glsl **Valor: normalizarVetor(calcularNormal(...))**

Significado: Vetor perpendicular a superficie, como uma flecha apontando para fora da parede ou face do cubo.

Como Explicar: Imagine um prego perpendicular espetado na madeira. Se o prego aponta para o teto, a face e horizontal. Se aponta para a direita, e uma parede lateral. E a bussola de cada face!

L (Vetor da Luz Normalizado)

Local: fragmentShad.glsl **Valor: normalizarVetor(lightDir)**

Significado: O vetor apontando em direcao ao Sol com comprimento matematico obrigatoriamente igual a 1.

Como Explicar: Reduzimos a forca da seta para exatamente 1 unidade de comprimento, para que ela guarde apenas a DIRECAO da luz, sem distorcer as contas de brilho.

diff (Fator Difuso / Brilho)

Local: fragmentShad.glsl **Valor: max(calcularIluminacao(N, L), 0.0)**

Significado: Numero entre 0.0 e 1.0 que diz quanta luz do Sol a superficie recebe pelo seu angulo de inclinacao.

Como Explicar: E a famosa Lei do Cosseno de Lambert: se a face aponta de frente para a luz, diff=1.0 (brilho maximo). Se estiver de lado, diff cai. Se estiver de costas, fica zero (sombra).

isSombra (Uniform booleano)

Local: mode3D.py / fragmentShad.glsl **Valor: 1 (renderizando sombra) ou 0**

Significado: Informa ao fragment shader que nao e para pintar o modelo com texturas, mas sim como uma silhueta escura translucida.

Como Explicar: Quando ativado, o shader ignora madeira e texturas e cospe a cor vec4(0.06, 0.06, 0.10, 0.45): um azul-marinho bem escuro com 45% de opacidade (55% transparente).

ALTURA_ELEVACAO (float)

Local: main.py

Valor: 0.25 (unidades do mundo)

Significado: Distancia vertical fixa que o objeto levita no ar quando o usuario clica e comeca a arrasta-lo com o mouse.

Como Explicar: Quando pegamos uma caixa no chao do mundo real, nos a erguemos. Elevar 0.25 unidades faz o objeto descolar do chao e revela a sombra embaixo dele, dando sensacao tatil de arrasto 3D.

alturaApoio (float)

Local: main.py / mode3D.py

Valor: 0.0 (chao) ou 1.15 (mesa/bancada)

Significado: Altura da superficie mais proxima logo abaixo do objeto onde a sombra deve ser carimbada e onde ele vai pousar.

Como Explicar: Se a caixa estiver no chao, a sombra fica no chao (Y=0.0). Se estiver em cima da bancada, a sombra sobe para o tampo da mesa (Y=1.15). A sombra nunca afunda no vazio.

S.y = 0.0001 (Matriz de Escala)

Local: mode3D.py (renderSombra)

Valor: glm.scale(1.0, 0.0001, 1.0)

Significado: Fator de escala no eixo Y que achata a malha 3D completa do objeto contra o chao.

Como Explicar: A analogia do rolo compressor: pegamos o modelo 3D original e esmagamos sua altura ate virar uma folha quase plana. Assim, a sombra tem exatamente o formato do objeto!

3. A Matematica Explicada Sem Segredos e Sem Jargoes

Muitas pessoas travam quando ouvem palavras como vetores, produto escalar ou derivadas parciais. Mas a intuicao por tras e incrivelmente simples quando usamos analogias do mundo real:

A. O que e um Vetor e a Funcao normalizarVetor(A)

Em Computacao Grafica, um vetor tridimensional vec3(x, y, z) nada mais e do que uma FLECHA que sai da origem e aponta para algum lugar no espaco. Ela possui uma DIRECAO e um COMPRIMENTO (tamanho da flecha).

Quando queremos saber apenas para onde o Sol esta apontando, o tamanho da flecha atrapalha as multiplicacoes. A nossa funcao normalizarVetor divide a flecha pelo seu proprio tamanho sqrt(x^2 + y^2 + z^2). O resultado e garantidamente um vetor de tamanho 1.0 (Unitario), com trava contra divisao por zero!

B. A Normal da Superficie e a Funcao calcularNormal(A, B)

Para saber se uma parede esta recebendo sol, a GPU precisa saber para onde aquela parede esta olhando (Vetor Normal N). Pense em um prego apontando 90 graus para fora da madeira.

Ao inves de usar o built-in cross, criamos a funcao calcularNormal(A, B) que aplica a formula classica do Produto Vetorial: multiplicamos cruzadamente os eixos Y e Z, Z e X, X e Y. O resultado e garantidamente uma seta perpendicular a face!

> **O Truque de Mestre: dFdx e dFdy (Sem precisar criar buffers de normais)**
>
> Em codigos mais antigos e pesados, o programador precisava cadastrar a normal de cada vertice na mao. No nosso projeto moderno, usamos dFdx(worldPos) e dFdy(worldPos). A GPU compara a posicao no espaco de pixels vizinhos na tela e passa para a nossa funcao calcularNormal, descobrindo o relevo instantaneamente!

C. Produto Escalar com a Funcao calcularIluminacao(A, B)

Como a GPU calcula o brilho diff = max(calcularIluminacao(N, L), 0.0)? Essa e a famosa Lei de Lambert e pode ser explicada assim:

- Luz de Frente (Angulo 0 graus): Aponte uma lanterna reta contra a parede. O facho fica concentrado em um circulo pequeno e ultra brilhante. O produto escalar entre N e L vale 1.0 (brilho maximo).
- Luz Inclinada (Angulo 45 a 60 graus): Incline a lanterna. O circulo de luz se espalha em uma elipse maior e a parede fica menos brilhante. O produto escalar diminui para algo entre 0.7 e 0.3.
- Luz Rasante ou de Costas (90 a 180 graus): Se a luz vier de lado ou por tras da parede, nenhuma luz atinge a face frontal. A nossa funcao calcularIluminacao daria zero ou um numero negativo. O comando max(..., 0.0) impede que o brilho fique negativo e crava em 0.0 (lado de sombra).

D. A Equacao Final de Iluminacao do Pixel

A cor de cada ponto no nosso shader e calculada por uma formula simples de adicao e multiplicacao:

- Luz Total: corFinal = (ambientColor + lightColor * diff) * texColor
- Em bom portugues: Pegamos a claridade minima que nunca zera (ambientColor), somamos com a luz do Sol proporcional ao angulo (lightColor * diff), e multiplicamos o resultado pela cor da textura da madeira ou da caixa. Resultado: faces iluminadas ganham vida e faces opostas ganham sombra suave!

4. O Mecanismo de Sombra Planar Projetada e Elevacao

Criar sombras em Computacao Grafica pode ser um dos topicos mais complexos (como Shadow Mapping com buffers de profundidade). Para o nosso projeto, escolhemos a tecnica de Sombra Planar Projetada, que e elegante, extremamente eficiente e visualmente impecavel.

A. Como funciona o Carimbo da Sombra?

> 1. Mesma Geometria: Ao inves de tentar desenhar um circulo generico, pegamos o proprio modelo 3D do objeto (com todos os seus triangulos e curvas).
> 2. O Rolo Compressor (Escala Y = 0.0001): Aplicamos uma matriz de escala que reduz a altura do modelo a quase zero. O objeto 3D vira uma estampa plana encostada no solo.
> 3. Silhueta Escura (isSombra = 1): Mandamos o shader desenhar essa estampa com cor azul-escura suave e 45% de opacidade, com blend transparente ativado (glBlendFunc).
> 4. Anti Z-Fighting (+0.005): Posicionamos a sombra 5 milimetros acima do chao real (yChao + 0.005). Isso impede que a sombra e o piso disputem o mesmo pixel na tela (efeito de piscar).

B. O Efeito de Elevacao Visual (ALTURA_ELEVACAO = 0.25)

Quando o usuario clica e arrasta um objeto pelo cenario, somamos 0.25 unidades na sua posicao Y. O objeto sobe no ar, enquanto a sombra continua projetada perfeitamente no chao ou no tampo da mesa (alturaApoio). Isso cria um efeito visual fantastico: a pessoa que joga tem a certeza absoluta de que pegou o item no colo e sabe exatamente em qual ponto do chao ele vai aterrissar quando soltar o botao do mouse!

5. Roteiro Pratico para Apresentar aos Colegas (2 Minutos)

Use este passo a passo caso va demonstrar o projeto em sala de aula ou em reuniao de equipe:

> Passo 1 (Introducao): Reparem como o cenario 3D ganha volume quando as faces recebem luz de acordo com a sua inclinacao em relacao ao Sol virtual.
> Passo 2 (Luz Ambiente vs Sol): Usamos duas luzes: uma luz ambiente suave (para as costas dos moveis nao ficarem pretas) e uma luz solar direcional calculada em tempo real na GPU pela Lei de Lambert.
> Passo 3 (Funcoes Proprias): Ao inves de usar funcoes prontas da GPU, escrevemos nossas proprias funcoes calcularNormal (produto vetorial), calcularIluminacao (produto escalar) e normalizarVetor (modulo unitario) para demonstrar a matematica pura.
> Passo 4 (Demonstracao do Arrasto): Ao clicar e arrastar qualquer objeto da mesa, vejam que ele levita um pouco (ALTURA_ELEVACAO = 0.25). Imediatamente vemos a sombra projetada no chao com o formato exato da peca!
> Passo 5 (Conclusao Tecnica): A sombra usa a tecnica planar projetada com matriz de escala Y minima, e a iluminacao calcula normais por derivadas de tela com nossa funcao manual, mantendo o codigo leve, limpo e profissional.

Resumo Final em Uma Frase

A iluminacao diz para o olho onde a peca esta virada; a sombra diz para o olho onde a peca esta pisando!